

TOY ISA 4

Instruction Set Architecture Specification

Proteus Lab

Generated on: 2026-09-17

Version: 1.4

Table of Contents

Architecture Overview	3
LD	4
ADD	5
BEQ	6
LI	7
ST	8
STP	9
ADDI	10
J	11
LD (post-index)	12
NOR	13
SSAT	14
RBIT	15
SYSCALL	16
BEXT	17
USAT	18

Architecture Overview

Register file and ABI

X0 return value	X1 arg 0	X2 arg 1	X3 arg 2
X4 arg 3	X5 arg 4	X6 arg 5	X7 arg 6
X8 syscall number	X9	X10	X11
X12	X13	X14	X15
X16	X17	X18	X19
X20	X21	X22	X23
X24	X25	X26	X27
X28	X29	X30	X31 LR

32 registers, 32-bit each

PC program counter

Memory

Memory is byte-addressed and little-endian. A word is 32 bits wide.

LD

Encoding

31:26	25:21	20:16	15:14	13:0
010111	base	rt	00	imm

Assembler

```
LD rt, [base, #imm]
```

Semantics

```
addr ← X[base] + sign_extend(imm)
```

```
raise MisalignedAccess unless isAligned(addr)
```

```
X[rt] ← memory[addr]
```

ADD

Encoding

31:26	25:21	20:16	15:11	10:6	5:0
000000	rs	rt	rd	00000	011000

Assembler

```
ADD rd, rs, rt
```

Semantics

```
 $X[rd] \leftarrow X[rs] + X[rt]$ 
```

BEQ

Encoding

31:26	25:21	20:16	15:0
001101	rs	rt	offset

Assembler

```
BEQ rs, rt, #offset
```

Semantics

```
target ← sign_extend(offset) << 2
```

```
cond ← X[rs] == X[rt]
```

```
PC ← if (cond) PC + target else PC + 4
```

LI

Encoding

31:26	25:21	20:16	15:0
101011	00000	rt	imm

Assembler

```
LI rt, #imm
```

Semantics

```
X[rt] ← sign_extend(imm)
```

ST

Encoding

31:26	25:21	20:16	15:14	13:0
101100	base	rt	00	imm

Assembler

```
ST rt, [base, #imm]
```

Semantics

```
addr ← X[base] + sign_extend(imm)
```

```
raise MisalignedAccess unless isAligned(addr)
```

```
memory[addr] ← X[rt]
```

STP

Encoding

31:26	25:21	20:16	15:11	10:0
111111	base	rt1	rt2	offset

Assembler

```
STP rt1, rt2, offset(base)
```

Semantics

```
addr ← X[base] + sign_extend(offset)
```

```
raise MisalignedAccess unless isAligned(addr)
```

```
memory[addr] ← X[rt1]
```

```
memory[addr + 4] ← X[rt2]
```

ADDI

Encoding

31:26	25:21	20:16	15:0
001010	rs	rt	imm

Assembler

```
ADDI rt, rs, #imm
```

Semantics

```
 $X[rt] \leftarrow X[rs] + \text{sign\_extend}(imm)$ 
```

J

Encoding

31:26	25:0
110111	instr_index

Assembler

J target

Semantics

$PC \leftarrow (PC \ \& \ 0xF0000000) \mid (instr_index \ll 2)$

LD (post-index)

Encoding

31:26	25:21	20:16	15:14	13:0
000111	base	rt	10	imm

Assembler

```
LD rt, [base], #imm
```

Semantics

```
raise MisalignedAccess unless isAligned(X[base])
```

```
X[rt] ← memory[X[base]]
```

```
X[base] ← X[base] + sign_extend(imm)
```

NOR

Encoding

31:26	25:21	20:16	15:11	10:6	5:0
000000	rs	rt	rd	00000	101001

Assembler

```
NOR rd, rs, rt
```

Semantics

```
 $X[rd] \leftarrow X[rs] \text{ nor } X[rt]$ 
```

SSAT

Encoding

31:26	25:21	20:16	15:11	10:0
010100	rd	rs	imm5	00000000000

Assembler

```
SSAT rd, rs, #imm5
```

Semantics

```
 $X[rd] \leftarrow \text{saturate\_signed}(X[rs], \text{imm5})$ 
```

Notes

Saturation of the signed value in $x[rs]$ to N bits, where $N = \#imm$

RBIT

Encoding

31:26	25:21	20:16	15:6	5:0
000000	rd	rs	0000000000	111110

Assembler

```
RBIT rd, rs
```

Semantics

```
 $X[rd] \leftarrow \text{reverse\_bits}(X[rs])$ 
```

Notes

Reverses the order of the bits in the $X[rs]$ register.

SYSCALL

Encoding

31:26	25:6	5:0
000000	code	010000

Assembler

SYSCALL

Semantics

SigException(SystemCall)

Notes

x8 — system call number, x0 - x7 — args, x0 — result, see man syscall

BEXT

Encoding

31:26	25:21	20:16	15:11	10:6	5:0
000000	rd	rs1	rs2	00000	100110

Assembler

```
BEXT rd, rs1, rs2
```

Semantics

```
 $X[rd] \leftarrow \text{bit\_extract}(X[rs1], X[rs2])$ 
```

Notes

Extracts and packs bits from register $X[rs1]$ into the positions where the corresponding bits in mask $X[rs2]$ are set. The extracted bits will place the least significant bits of the result Rd .

USAT

Encoding

31:26	25:21	20:16	15:11	10:0
110000	rd	rs	imm5	000000000000

Assembler

```
USAT rd, rs, #imm5
```

Semantics

```
 $X[rd] \leftarrow \text{saturate\_unsigned}(X[rs], \text{imm5})$ 
```

Notes

Saturation of the unsigned value in $X[rs]$ to N bits, where $N = \#imm$